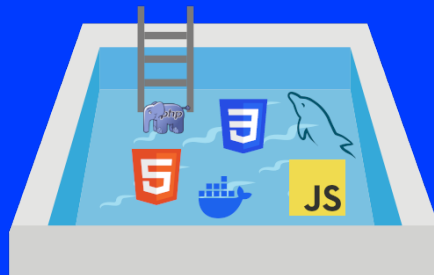




BOOTCAMP WEB

< DAY 01 - SHELL BASICS />



BOOTCAMP WEB

Welcome to the web bootcamp.

Before writing a single line of HTML, you'll spend today with the two tools you'll reach for every single day from now on: a [terminal](#) and its [command line](#).

Every day of this bootcamp starts the same way — open a terminal, create the directory for the day, run a command — so the time you invest today pays off on every day that follows.

Today's tasks are designed to help you take your first steps with these two common tools.



Try to complete today's tasks using only your keyboard and a terminal (on Windows, use [WSL](#)).
Use your mouse or touchpad as little as possible.
It may be challenging at first, but it will greatly boost your future efficiency.

Task01

Delivery: `./task01/readme`

In a directory named `task01`, create a file `readme` and writes inside "Hello World!".



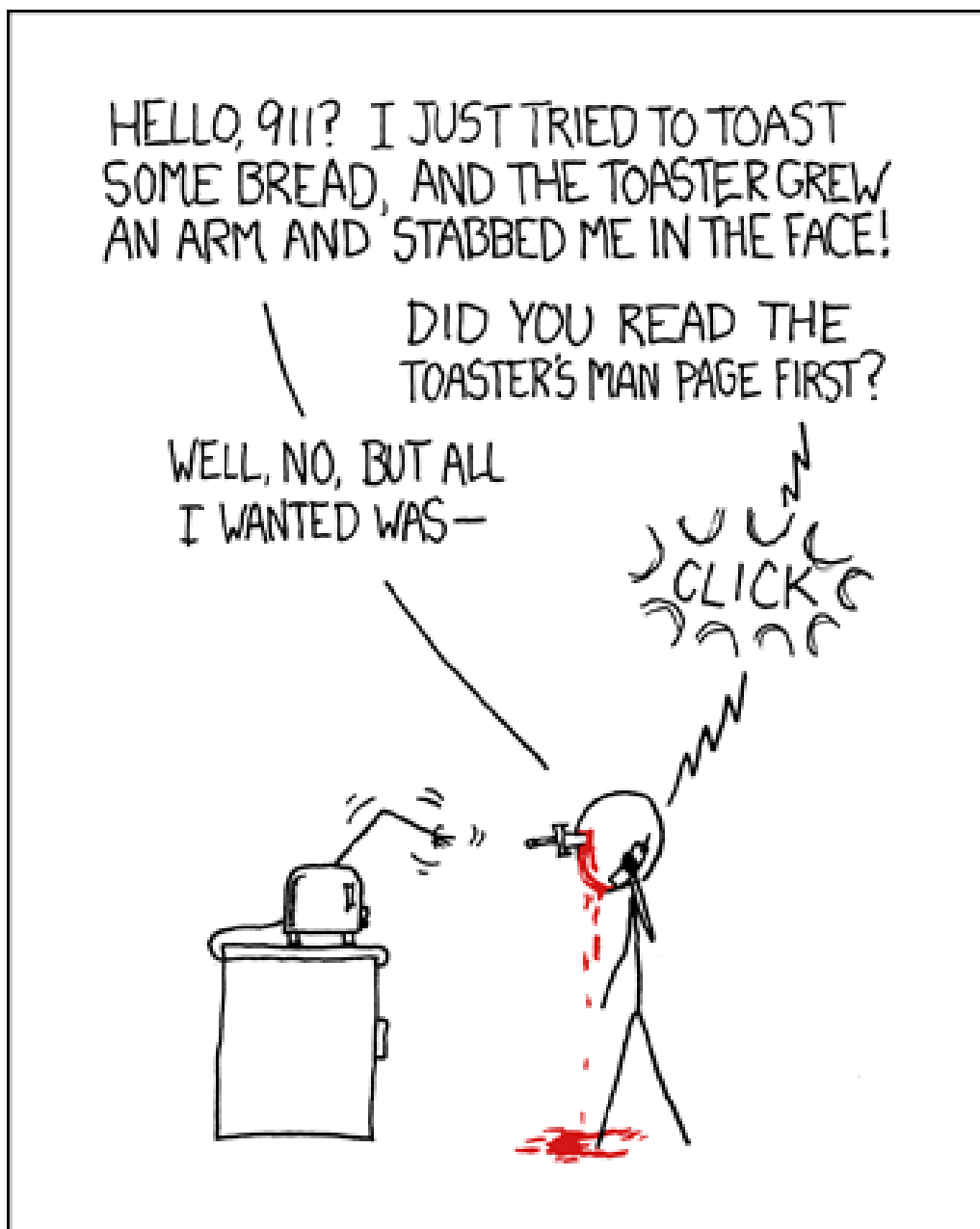
You can create a file with a shell command, you can even write into or read from a file with shell commands. Try to discover and use them.

```
~> cat -e task01/readme  
Hello World!
```

Task02

Delivery: `./task02/howto`

Write in the file `task02/howto` the shell command that displays the manual page of the shell command used to display manual pages.



Task03

Delivery: `./task03/listing`

Write in the file `task03/listing` one shell command that displays a list of the files and directories contained in the current working directory. This list must:

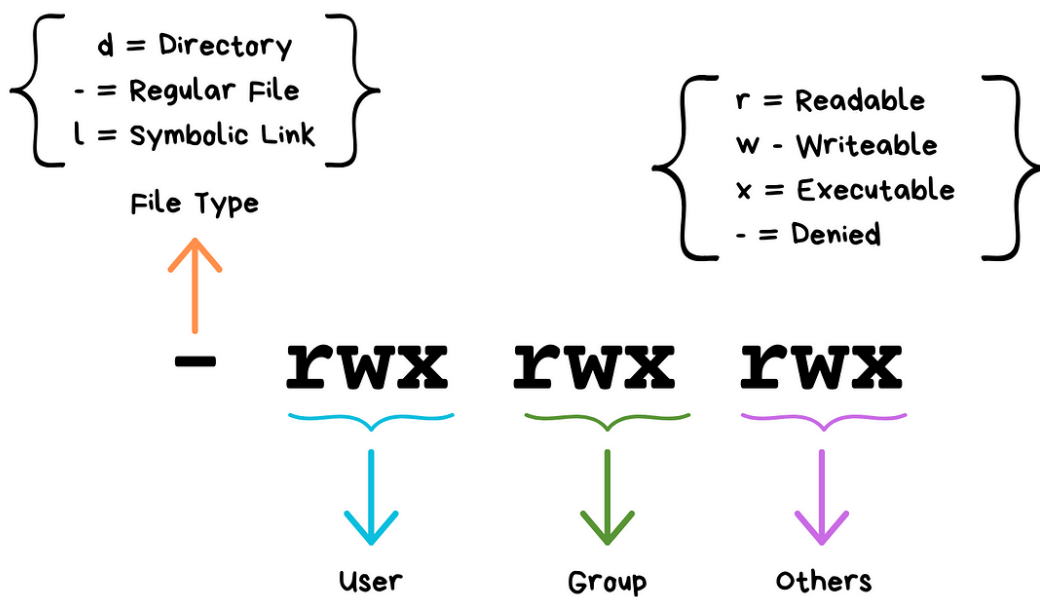
- ✓ be **human-readable**;
- ✓ have a **long format**;
- ✓ contains **all** files and directories (including the hidden ones).



Find the right command. Then, read its manual to check the available options.



Now, take a time to fully understand the meaning of the first ten characters of each line displayed by your command. You will need this for your next tasks.



For the next tasks (and days), we won't repeat the file location and name in the instructions. Please refer to the specified naming convention.

Task04

Delivery: `./task04/create_file.sh`

Submit a Bash shell script which creates a file `toto.txt` in the *current directory*.



The *current directory* is the directory from which the script is executed.



Each time you create a script, remember to:

- ✓ start with the **shebang** line;
- ✓ make it executable by setting the right **permissions**.

Task05

Delivery: `./task05/write_in_file.sh`

Submit a Bash shell script which writes the word "overwrite" in a file named `toto.txt`.

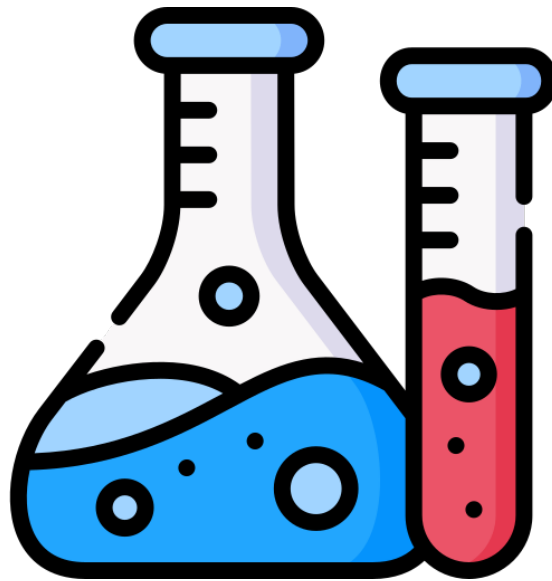


Find below some steps to test your current script:

- ✓ open a terminal in the current task directory;
- ✓ inside it, create `toto.txt` (by using the script from task04, for instance);
- ✓ then, run your current script;
- ✓ finally, read the content of `toto.txt` (by using the command from task01, for instance).



Tests ensure that a solution meets its requirements and behaves as expected.
It is good practice to test as much as possible.



Task06

Delivery: `./task06/create_dirs.sh`

Submit a Bash shell script that uses a single command to create a directory named "music" in the current directory, with two subdirectories named "house" and "techno" inside it.



Look for the right command and check its options in the manual.

Task07

Delivery: `./task07/search_in_file.sh`

Submit a Bash shell script that:

- ✓ takes two string parameters: the first is a pattern, and the second is a file path;
- ✓ searches for the pattern in the content of the file.



Check if one of your passwords may be easily compromised by running this script against [this file containing 10 millions of common passwords](#).



On a common laptop (i5/i7, 8-16Go RAM), it approximately takes between 30 seconds to 2 minutes to loop over 10 millions passwords.

Task08

Delivery: `./task08/batch_replace.sh`

Submit a Bash shell script which **finds** all files with a `.js` extension, and **executes** a command to **replace** the text "myMoule" with the text "myModule" inside each file.



Look for the right command and check its options in the manual.
Then, use a powerful stream editor.

Task09

Delivery: `./task09/conditional_backup.sh`

Submit a Bash shell script which displays "Welcome home!" only if it is launched from the [home directory](#) on Linux or macOS.



You should use the directory's environment variable **and** a logical operator or condition.

Task10

Delivery: `./task10/chain_of_commands.sh`

Submit a Bash shell script which:

- ✓ reads the file named `log.txt`;
- ✓ search for all occurrences of the pattern "error", in a case-insensitive manner;
- ✓ displays only the 5 first matches.



Look for each command, separately. Then, find an operator to redirect the output of one command to the input of another command.

Task11

Delivery: `./task11/loop_files.sh`

Submit a Bash shell script which:

- ✓ iterates over all Markdown files (with `.md` extension) in the current directory;
- ✓ appends the text `This is a new line.` at the end of each file.



Your script must NOT overwrite the content of each file but only ADD to it!

Task12

Delivery: `./task12/my_automate.sh`

Submit a Bash shell script which:

- ✓ takes two integer parameters: the first represents a total of tasks of a bootcamp day, represented by the second parameter;
- ✓ creates a directory dedicated to the day containing multiple subdirectories, each one dedicated to a task.



The directories must contain numbers made of 2 digits.
Remember the leading zeroes!

For instance:

```
~> ./my_automate.sh 12 3
~> tree -FQ --charset=ascii .
"/
|-- "day03"/
|   |-- "task01"/
|   |-- "task02"/
|   |-- "task03"/
|   |-- "task04"/
|   |-- "task05"/
|   |-- "task06"/
|   |-- "task07"/
|   |-- "task08"/
|   |-- "task09"/
|   |-- "task10"/
|   |-- "task11"/
|   |-- "task12"/
```

14 directories, 0 files

v 1.2

{EPITECH}